

Linear Data Structures

June 24, 2026

Content

- ▶ Linear Data Structures
- ▶ Arrays and Structures
- ▶ Linked Lists
 - Singly Linked List
 - Doubly Linked List
 - Circular Linked List
- ▶ Stacks and Queues
 - Array based Stack
 - Linked List based Stack
 - Array based Queue
 - Linked List based Queue
- ▶ Applications
 - Sorting (Few)
 - Linear Sort
 - Divide and Conquer

Linear Data Structures

- ▶ Data types such as integer (e.g. 2, -8), floating point (e.g. 2.4, -9.1), etc. represent different kinds of data that can be stored in a computer's memory.
- ▶ Various logical and arithmetic operations can be performed on these data types to produce desired results.
- ▶ An Abstract Data Type (ADT) is a mathematical or logical model for a data type that defines:
 - the type of data stored, and
 - the operations that can be performed on the data,without specifying how the data is implemented internally.
- ▶ *Example:* List with ops. like insert/replace/delete elements
 $S = [1, 2, 3, 7, -1]$

Arrays and Structures

- ▶ *Array*: Fixed-sized ordered collection of elements of the same data type stored in contiguous memory locations
`int a[100];`
- ▶ *Structure*: A collection of variables, possibly of different data types, grouped together under a single name and typically stored in contiguous memory locations.

More memory padding

```
typedef struct
{
    char  key;
    double value;
    char key1;
} dictionary;
```

`sizeof(dictionary)?`

Only single tail padding

```
typedef struct
{
    double value;
    char  key;
    char key1;
} dictionary;
```

`sizeof(dictionary)?`

Linked Lists

- ▶ *Linked List*: A dynamic data structure consisting of nodes connected by links, typically stored in non-contiguous memory locations.
- ▶ Node insertion and deletion makes the list dynamic

```
1      typedef struct Node {  
2          int data;  
3          struct Node* next;  
4      } Node_t;  
5  
6      Node_t* head = NULL;  
7      Node_t* n1 = (Node_t*)malloc(sizeof(Node_t));  
8      n1->data = 10;  
9      n1->next = NULL;  
10  
11     head = n1;
```

Linked Lists: Insertion at beginning

```
1 Node_t* insertAtPos(Node_t* head, int value, int pos)
2 {
3     Node_t* newNode = (Node_t*)malloc(sizeof(Node_t));
4     newNode->data = value;
5     // newNode will be the head from now if pos == 1
6     if (pos == 1) {
7         newNode->next = head;
8         head = newNode;
9     }
10    return head;
11 }
```

Linked Lists: Insertion at desired position

```
1 Node_t* insertAtPos(Node_t* head, int value, int pos)
2 {
3     Node_t* newNode = (Node_t*)malloc(sizeof(Node_t));
4     newNode->data = value;
5     // newNode will be the head from now if pos == 1
6     if (pos == 1) {
7         newNode->next = head;
8         head = newNode; return head;
9     }
10    Node_t* temp = head;
11    // Traversing the links to reach the position 'pos-1'
12    for (int i = 1; i < pos - 1 && temp != NULL; i++)
13        temp = temp->next;
14    // newNode's pointer points to pointer of the node at 'pos-1'
15    newNode->next = temp->next;
16    // Pointer of Node at position 'pos-1' points to newNode
17    temp->next = newNode;
18    return head;
19 }
```

Linked Lists: Insertion at end

```
1 Node_t* insertAtEnd(Node_t* head, int value)
2 {
3     Node_t* newNode = (Node_t*)malloc(sizeof(Node_t));
4     newNode->data = value;
5     newNode->next = NULL;
6     // Nothing to delete if no node exists
7     if (head == NULL)
8         return newNode;
9     Node_t* temp = head;
10    // Traverse the links to reach the tail node
11    while (temp->next != NULL)
12        temp = temp->next;
13    // Attach newnode as the last node
14    temp->next = newNode;
15    return head;
16 }
```


Linked Lists: Deletion by Value

```
1 Node_t* deleteValue(Node_t* head, int value)
2 {
3     Node_t *temp = head, *prev = NULL;
4     // If any node present and head node value == target value
5     if (temp != NULL && temp->data == value) {
6         head = temp->next; // new head will be following node
7         free(temp);        // delete the head (old)
8         return head;       // return the new head pointer
9     }
10    // Traverse and find the node with value == target value
11    while (temp != NULL && temp->data != value) {
12        prev = temp;
13        temp = temp->next;
14    }
15    if (temp == NULL) return head; // Not a valid one
16    prev->next = temp->next;
17    free(temp); // delete the matching node
18    return head;
19 }
```

Linked Lists: Deletion at end

```
1 Node_t* deleteEnd(Node_t* head) {
2     if (head == NULL)           // No node, skip the process
3         return NULL;
4
5     if (head->next == NULL) {    // One node, delete it
6         free(head);
7         return NULL;
8     }
9
10    Node_t* temp = head;
11
12    while (temp->next != NULL)    // Traverse till node before tail
13        temp = temp->next;
14
15    free(temp->next);             // delete the tail
16    temp->next = NULL;            // make node before tail as tail
17
18    return head;
19 }
```

List - Python

```
1 # Implementation in Python
2 List = [10, 20, 30, 40]
3
4 # Access (Start Index = 0)
5 print(List[2])           # 30
6
7 # Insert at end
8 List.append(50)          # [10, 20, 30, 40, 50]
9
10 # Insert at index
11 List.insert(2, 25)       # [10, 20, 25, 30, 40, 50]
12
13 # Delete
14 List.pop(3)              # [10, 20, 25, 40, 50]
15
16 # Search
17 idx = List.index(40)     # 3
```

List - Python

```
1 # Structure Equivalent
2 class Node:
3     def __init__(self, data): # constructor
4         # Attributes of Class "Node"
5         self.data = data
6         self.next = None
7 class LinkedList:
8     def __init__(self):
9         self.head = None
10 def insert_front(self, data):
11     node = Node(data)           # Object
12     node.next = self.head      # Next of new node as Head
13     self.head = node          # Head as new node
14 ll = LinkedList()              # Head node creation
15 ll.insert_front(30)
16 ll.insert_front(20)
17 ll.insert_front(10)           # [10, 20, 30]
```

List - Python

```
1 def insert(self, pos, data):
2     node = Node(data)          # Object
3     if pos == 0:
4         node.next = self.head  # Next of new node as Head
5         self.head = node       # Head as new node
6     temp = self.head
7     for i in range(pos-1):
8         temp = temp.next
9     node.next = temp.next
10    temp.next = node
11
12 # ll = [10, 20, 30]
13 ll.insert(1, 40)              # [10, 40, 20, 30]
```

List - Python

```
1 def delete(self, pos):
2     if self.head == None:
3         return
4     if self.head.next == None:
5         self.head = None
6     temp = self.head
7     for i in range(pos-1):
8         temp = temp.next
9     temp.next = temp.next.next
10
11 # ll = [10, 20, 30]
12 ll.insert(1, 40)          # [10, 40, 20, 30]
```

Linked Lists - Double sided

- ▶ *Linked List*: A dynamic data structure consisting of nodes connected by links, typically stored in non-contiguous memory locations.
- ▶ Node insertion and deletion from both sides allowed

```
1      typedef struct Node {  
2          int data;  
3          struct Node* next;  
4          struct Node* prev;  
5      } Node_t;  
6  
7      Node_t* head = NULL;  
8      Node_t* n1 = (Node_t*)malloc(sizeof(Node_t));  
9      n1->data = 10;  
10     n1->next = NULL;  
11     n1->prev = NULL;  
12  
13     head = n1;
```

Linked Lists - Double sided

```
1 Node_t* insertAtPos(Node_t* head, int value, int pos)
2 {
3     Node_t* newNode = (Node_t*)malloc(sizeof(Node_t));
4     newNode->data = value;
5     // newNode will be the head from now if pos == 1
6     if (pos == 1) {
7         head->prev = newNode;
8         newNode->next = head;
9         head = newNode;
10    }
11    return head;
12 }
```


Linked Lists (Double sided): Insertion at desired position

```
1 Node_t* insertAtPos(Node_t* head, int value, int pos)
2 {
3     Node_t* newNode = (Node_t*)malloc(sizeof(Node_t));
4     newNode->data = value;
5     if (pos == 1) {
6         head->prev = newNode;
7         newNode->next = head;
8         head = newNode; return head;
9     }
10    Node_t* temp = head;
11    for (int i = 1; i < pos - 1 && temp != NULL; i++)
12        temp = temp->next;
13    Node_t* temp1 = temp->next;
14    newNode->next = temp1;
15    newNode->prev = temp;
16    temp1->prev = newNode;
17    temp->next = newNode;
18
19    return head;
```

Linked Lists (Double sided): Insertion at end

```
1 Node_t* insertAtEnd(Node_t* head, int value)
2 {
3     Node_t* newNode = (Node_t*)malloc(sizeof(Node_t));
4     newNode->data = value;
5     newNode->next = NULL;
6     // Nothing to delete if no node exists
7     if (head == NULL)
8         return newNode;
9     Node_t* temp = head;
10    // Traverse the links to reach the tail node
11    while (temp->next != NULL)
12        temp = temp->next;
13    // Attach newnode as the last node
14    temp->next = newNode;
15    newNode->prev = temp;
16    return head;
17 }
```

Linked Lists (Double sided): Deletion by Value

```
1 Node_t* deleteValue(Node_t* head, int value)
2 {
3     Node_t *temp = head, *prev = NULL;
4     if (temp != NULL && temp->data == value) {
5         head = temp->next; // new head will be following node
6         if (head != NULL) head->prev = NULL;
7         free(temp);      // delete the head (old)
8         return head;     // return the new head pointer
9     }
10    while (temp != NULL && temp->data != value) {
11        prev = temp;
12        temp = temp->next;
13    }
14    if (temp == NULL)      return head;    // Not a valid one
15    prev->next = temp->next;
16    if (temp->next != NULL) temp->next->prev = prev;
17    free(temp);           // delete the matching node
18    return head;
19 }
```

Linked Lists (Double sided): Deletion at end

```
1 Node_t* deleteEnd(Node_t* head) {  
2     if (head == NULL)           // No node, skip the process  
3         return NULL;  
4     if (head->next == NULL || head->prev == NULL) {  
5         free(head);  
6         return NULL;  
7     }  
8     Node_t* temp = head;  
9     while (temp->next != NULL)  // Traverse till tail forward dir.  
10         temp = temp->next;  
11     if (temp->prev != NULL)  
12         temp->prev->next = NULL; // Target node removed  
13     free(temp);                // free the memory  
14     return head;  
15 }
```

Linked Lists (Double sided) - Python

```
1 class Node:
2     def __init__(self, data):
3         self.data = data
4         self.prev = None
5         self.next = None
6 class DoublyLinkedList:
7     def __init__(self):
8         self.head = None
9 def append(self, data):
10     new_node = Node(data)           # new node creation
11     if self.head is None:           # List is empty
12         self.head = new_node        # Keep new node as Head
13         return
14     # List is not empty
15     current = self.head              # Temp. node
16     while current.next:               # Traverse till end
17         current = current.next
18     current.next = new_node           # new node as last node
19     new_node.prev = current           # prev. of new last as old last
```

Linked Lists (Double sided) - Python

```
1 # Insert
2 def insert_at_index(self, index, data):
3     new_node = Node(data)
4     if index == 0: # new node as Head
5         new_node.next = self.head # make head as second node
6         if self.head:             # if head exists (not None)
7             # Assign previous of head as new node
8             self.head.prev = new_node
9             # Keep head as new node
10            self.head = new_node
11            return
12    # continues
```

Linked Lists (Double sided) - Python

```
1      current = self.head          # Start Traversal to find the index
2      position = 0
3      while current and position < index - 1:
4          current = current.next # Traversal
5          position += 1
6      new_node.next = current.next # same as C impl.
7      new_node.prev = current      # same as C impl.
8      if current.next:
9          current.next.prev = new_node # same as C impl.
10     current.next = new_node       # same as C impl.
11     # continues
```

Linked Lists (Double sided) - Python

```
1 # Delete by index
2 def delete_at_index(self, index):
3     if self.head is None:           # Empty list
4         return
5     if index == 0:
6         self.head = self.head.next
7     if self.head:                   # Non-empty list after delete
8         self.head.prev = None      # Double-sided, prev. as None
9         return
10    current = self.head
11    position = 0
12    while current and position < index:
13        current = current.next      # Traversal to find delete node
14        position += 1
15    # Rewiring after delete
16    if current.prev:                 # same as C impl.
17        current.prev.next = current.next
18    if current.next:                 # same as C impl.
19        current.next.prev = current.prev
```


Linked Lists - Circular

- ▶ *Linked List*: A dynamic data structure consisting of nodes connected by links, typically stored in non-contiguous memory locations.
- ▶ Tail Node will be linked to Head Node

```
1      typedef struct Node {  
2          int data;  
3          struct Node* next;  
4          struct Node* prev;  
5      } Node_t;  
6  
7      Node_t* head = NULL;  
8      Node_t* n1 = (Node_t*)malloc(sizeof(Node_t));  
9      n1->data = 10;  
10     n1->next = n1;  
11     n1->prev = n1;  
12  
13     head = n1;
```

Linked Lists - Circular

```
1 Node_t* insertAtPos(Node_t* head, int value, int pos)
2 {
3     Node_t* newNode = (Node_t*)malloc(sizeof(Node_t));
4     newNode->data = value;
5     // newNode will be the head from now if pos == 1
6     if (pos == 1) {
7         head->prev = newNode;
8         head->next = newNode;
9         newNode->next = head;
10        newNode->prev = head;
11        head = newNode;
12    }
13    return head;
14 }
```

Linked Lists (Circular): Insertion at desired position

```
1 Node_t* insertAtPos(Node_t* head, int value, int pos)
2 {
3     Node_t* newNode = (Node_t*)malloc(sizeof(Node_t));
4     newNode->data = value;
5     if (pos == 1) {
6         head->prev = newNode;
7         head->next = newNode;
8         newNode->next = head;
9         newNode->prev = head;    return head;
10    }
11    Node_t* temp = head;
12    for (int i = 1; i < pos - 1; i++)
13        temp = temp->next;
14    Node_t* temp1 = temp->next;
15    newNode->next = temp1;
16    newNode->prev = temp;
17    temp1->prev = newNode;
18    temp->next = newNode;
19
```

Linked Lists (Circular): Insertion at end

```
1 Node_t* insertAtEnd(Node_t* head, int value)
2 {
3     Node_t* newNode = (Node_t*)malloc(sizeof(Node_t));
4     newNode->data = value;
5     newNode->next = head;
6     Node_t* temp = head->prev          // Last node
7     temp->next = newNode;
8     newNode->prev = temp;
9     head->prev = newNode;
10    return head;
11 }
```

Linked Lists (Circular): Deletion by Value

```
1 Node_t* deleteValue(Node_t* head, int value)
2 {
3     Node_t *temp = head, *prev = NULL;
4     if (temp->data == value) {
5         head = temp->next; // new head will be following node
6         head->prev = temp->prev;
7         free(temp);        // delete the head (old)
8         return head;       // return the new head pointer
9     }
10    // If loop continues till tail, then temp will be head
11    while (temp->data != value && temp->next != head) {
12        prev = temp;
13        temp = temp->next;
14    }
15    prev->next = temp->next;
16    temp->next->prev = prev;
17    free(temp);    // delete the matching node
18    return head;
19 }
```

Linked Lists (Circular): Deletion at end

```
1 Node_t* deleteEnd(Node_t* head) {  
2     if (head == NULL)           // No node, skip the process  
3         return NULL;  
4     Node_t* temp = head->prev;  
5     temp->prev->next = head; // Target node removed  
6     head->prev = temp->prev;  
7     free(temp);           // free the memory  
8     return head;  
9 }  
10  
11 Activity: Try Python implementation of circular linked list
```

Linear Data Structures

- ▶ *Complexity*: A way to describe how the resources (time, memory) required by specific operation grow as the input size grows.
- ▶ The two most common resources measured are:
 - *Time complexity*: How the running time grows.
 - *Space complexity*: How the memory usage grows.
- ▶ Asymptotic notation, especially Big $\mathcal{O}$ notation will be shorthand to express the complexity.
- ▶ *Big $\mathcal{O}$ Notation*: Big $\mathcal{O}$ describes an upper bound on complexity (worst-case complexity). E.g.: $5 \log n + 2n \leq 5n + 2n = \mathcal{O}(n)$

Linear Data Structures

► Common Big $\mathcal{O}$ s:

Table: Big $\mathcal{O}$

Big $\mathcal{O}$	Name	Relevance
$\mathcal{O}(1)$	constant	Element Access
$\mathcal{O}(n)$	Linear	Linear Search
$\mathcal{O}(\log n)$	Logarithmic	Binary Search
$\mathcal{O}(n^2)$	Quadratic	Bubble Sort
$\mathcal{O}(n^3)$	Cubic	-
$\mathcal{O}(n \log n)$	Linearithmic	Merge Sort
$\mathcal{O}(2^n)$	Exponential	-
$\mathcal{O}(n!)$	Factorial	Permutation Sort

► Note: For **large** n ,

$$\mathcal{O}(n!) > \mathcal{O}(2^n) > \mathcal{O}(n^3) > \mathcal{O}(n^2) > \mathcal{O}(n \log n) > \mathcal{O}(n) > \mathcal{O}(\log n) > \mathcal{O}(1)$$

Linear Data Structures

- *Node*: For ' n ' nodes, LL ($4n + 4n = 8n$) bytes and Array ($4n$ bytes) require $\mathcal{O}(n)$

Table: Computational complexity of Array and Linked List

Operation	Time Complexity			Space Complexity		
	Array	Singly LL	Doubly / Circular LL	Array	Singly LL	Doubly / Circular LL
Access	$\mathcal{O}(1)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$
Search	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$
Insert	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$
Delete	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$
Insert at end	$\mathcal{O}(1)$	$\mathcal{O}(n)$	$\mathcal{O}(1)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$
Delete at end	$\mathcal{O}(1)$	$\mathcal{O}(n)$	$\mathcal{O}(1)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$
Insert at start	$\mathcal{O}(n)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$
Delete at start	$\mathcal{O}(n)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$

Linked Lists - Stacks

- ▶ *Stack*: A linear data structure that follows the Last-In, First-Out (LIFO) principle, where insertion and deletion occur only at one end called the top.
- ▶ A stack may be implemented using arrays or linked lists. Common operations include pop, push, peek.
- ▶ Tail will be the Stack 'Top' (Below implementation)

```
1      typedef struct Node {  
2          int data;  
3          struct Node* next;  
4          struct Node* prev;  
5      } Node_t;  
6      Node_t* top = NULL;  
7      Node_t* newNode = (Node_t*)malloc(sizeof(Node_t));  
8      newNode->data = 10;  
9      newNode->next = newNode;  
10     newNode->prev = newNode;
```

Linked Lists (Stacks): Insertion at end - Push

```
1 Node_t* top = NULL;
2 void push(int value)
3 {
4     Node_t* newNode = (Node_t*)malloc(sizeof(Node_t));
5     newNode->data = value;
6     if (top == NULL) {
7         newNode->next = newNode;
8         newNode->prev = newNode;
9         top = newNode;
10        return;
11    }
12    Node_t* head = top->next;
13    newNode->next = head;
14    newNode->prev = top;
15    top->next = newNode;
16    head->prev = newNode;
17    top = newNode;
18 }
```

Linked Lists (Stacks): Deletion at end - Pop

```
1 Node_t* top = NULL;
2 void pop() {
3     if (top == NULL)           // No node, skip the process
4         return;
5     if (top->next == top) {
6         free(top);
7         top = NULL;
8         return;
9     }
10    Node_t* head = top->next;
11    Node_t* temp = top->prev;
12    temp->next = head;
13    head->prev = temp;
14    Note_t* temp1 = top;
15    top = temp;
16    free(temp1);    // free the memory
17 }
```

Stacks - Python

```
1 class Stack:
2     def __init__(self):
3         self.items = []
4     def push(self, item):
5         self.items.append(item)
6     def pop(self):
7         if self.is_empty():
8             raise IndexError("Empty")
9         return self.items.pop()
10    def peek(self):
11        if self.is_empty():
12            raise IndexError("Empty")
13        return self.items[-1]
14    def is_empty(self):
15        return len(self.items) == 0
16    def size(self):
17        return len(self.items)
```

Linked Lists - Queues

- ▶ *Queue*: A linear data structure that follows the First-In, First-Out (FIFO) principle, where insertion occurs at the rear and deletion occurs at the front.
- ▶ A queue can be implemented using arrays or linked lists. Common operations include enqueue, dequeue, peek.



```
1      typedef struct Node {  
2          int data;  
3          struct Node* next;  
4          struct Node* prev;  
5      } Node_t;  
6      Node_t* top = NULL;  
7      Node_t* newNode = (Node_t*)malloc(sizeof(Node_t));  
8      newNode->data = 10;  
9      newNode->next = newNode;  
10     newNode->prev = newNode;
```

Linked Lists (Queues): Insertion at tail - Enqueue

```
1 Node_t* top = NULL;
2 void enqueue(int value)
3 {
4     Node_t* newNode = (Node_t*)malloc(sizeof(Node_t));
5     newNode->data = value;
6     if (top == NULL) {
7         newNode->next = newNode;
8         newNode->prev = newNode;
9         top = newNode;
10        return;
11    }
12    Node_t* head = top->next;
13    newNode->next = head;
14    newNode->prev = top;
15    top->next = newNode;
16    head->prev = newNode;
17    top = newNode;
18 }
```

Linked Lists (Queues): Deletion at head - Dequeue

```
1 Node_t* top = NULL;
2 void dequeue() {
3     if (top == NULL)           // No node, skip the process
4         return;
5     if (top->next == top) {
6         free(top);
7         top = NULL;
8         return;
9     }
10    Node_t* head = top->next;
11    Node_t* newhead = head->next;
12    newhead->prev = top;
13    top->next = newhead;
14    free(head);    // free the memory
15 }
```


Array - Stacks

► *Stacks using Array:*

```
1 #define MAX 100
2 int stack[MAX];
3 int top = -1;
4
5 void push(int x) {
6     if (top == MAX - 1) {
7         return;
8     }
9     stack[++top] = x;
10 }
11
12 void pop() {
13     if (top == -1) {
14         return;
15     }
16     top--;
17 }
```

Array - Queues

► Queues using Array:

```
1 #define MAX 100
2 int queue[MAX];
3 int front = -1, rear = -1;
4 void enqueue(int x) {
5     if (rear == MAX - 1)    return;
6     if (front == -1)        // front > rear in dequeue
7         front = 0;
8     queue[++rear] = x;
9 }
10 void dequeue() {
11     if (front == -1 || front > rear) {
12         return;
13     }
14     front++;
15     if (front > rear)
16         front = rear = -1;
17 }
```

Stacks and Queues

► *Time to Think:*

- I want to pick an element out of a stack without disturbing the order. *Hint:* Use an extra stack
- Need to check whether parenthesis, curly braces and brackets are balanced or not in my program using stack. *Hint:* Conditionally use 'push' and 'pop'
- Reverse the list using stack-queue combination (Single stack and queue only allowed),
List = [1, 2, 7, 3], Reverse = [3, 7, 2, 1]
- Mirror the list using stack-queue combination (Single stack and queue only allowed)
List = [1, 2, 7, 3], Reverse = [1, 2, 7, 3, 3, 7, 2, 1]
- Multiple copying list elements using a single queue and its relevant operations.
List = [1, 2, 7, 3], 3-copy List = [1, 1, 1, 2, 2, 2, 7, 7, 7, 3, 3, 3]

Linear Data Structures

- *Space Complexity:* $\mathcal{O}(n)$

Table: Time complexity of Stacks and Queues

Operation	Time Complexity			
	Stack (Array)	Queue (Array)	Stack (LL)	Queue (LL)
Access/Peek	$\mathcal{O}(n)/\mathcal{O}(1)$	$\mathcal{O}(n)$	$\mathcal{O}(n)/\mathcal{O}(1)$	$\mathcal{O}(n)$
Search	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$
Push/Enqueue	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$
Pop/Dequeue	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$

Applications - Sorting

- *Sorting*: The process of arranging elements of a linear data structure in ascending or descending order. Commonly operate on linear data structures such as arrays and linked lists.

```
1  /***** Using Array *****/
2  void bubbleSort(int arr[], int n) {
3      for (int i = 0; i < n - 1; i++) { // n-1 iterations
4          // n-1, n-2, ..., 1 iteration
5          // 1st, 2nd, ..., last max/min
6          for (int j = 0; j < n - i - 1; j++) {
7              if (arr[j] > arr[j + 1]) {
8                  int temp = arr[j];
9                  arr[j] = arr[j + 1];
10                 arr[j + 1] = temp;
11             }
12         }
13     }
14 }
```

Applications - Sorting

```
1  /***** Using Linked List *****/
2  void bubbleSort(struct Node* head) {
3      int swapped;
4      Node_t *ptr1, *lptr = NULL;
5      if (head == NULL) return;
6      do {
7          swapped = 0; ptr1 = head;
8          while (ptr1->next != lptr) {
9              if (ptr1->data > ptr1->next->data) {
10                 int temp = ptr1->data;
11                 ptr1->data = ptr1->next->data;
12                 ptr1->next->data = temp;
13                 swapped = 1;
14             }
15             ptr1 = ptr1->next;
16         }
17         lptr = ptr1; // last sorted node
18     } while (swapped);
19 }
```

Applications - Sorting

► *Sorting*: Divide-and-conquer sorting

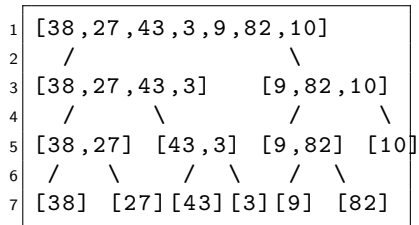
```
1  /***** Merge Sort *****/
2  void merge(int arr[], int left, int mid, int right) {
3      int i = left, j = mid + 1, k = 0;
4      int temp[right - left + 1];
5      while (i <= mid && j <= right) {
6          if (arr[i] < arr[j])
7              temp[k++] = arr[i++];
8          else
9              temp[k++] = arr[j++];
10     }
11     while (i <= mid)
12         temp[k++] = arr[i++];
13     while (j <= right)
14         temp[k++] = arr[j++];
15     for (i = left, k = 0; i <= right; i++, k++)
16         arr[i] = temp[k];
17 }
```

Applications - Sorting

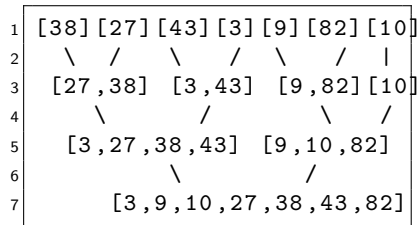
```
1  ***** Merge Sort *****
2  void mergeSort(int arr[], int left, int right) {
3      if (left < right) {
4          int mid = (left + right) / 2;
5          // Divide and conquer recursively
6          mergeSort(arr, left, mid);
7          mergeSort(arr, mid + 1, right);
8          // Sorting with smaller units and Combining
9          merge(arr, left, mid, right);
10     }
11 }
```


Applications - Sorting

Divide



Combine



Applications - Sorting

► *Sorting:* Divide-and-conquer sorting

```
1 (1) MergeSort(0,6)
2     (2) MergeSort(0,3)
3         (3) MergeSort(0,1)
4             (4) MergeSort(0,0)
5             (5) MergeSort(1,1)    (6) Merge(0,0,1)
6         (7) MergeSort(2,3)
7             (8) MergeSort(2,2)
8             (9) MergeSort(3,3)    (10) Merge(2,2,3)
9         (11) Merge(0,1,3)
10    (12) MergeSort(4,6)
11        (13) MergeSort(4,5)
12            (14) MergeSort(4,4)
13            (15) MergeSort(5,5)    (16) Merge(4,4,5)
14        (17) MergeSort(6,6)    (18) Merge(4,5,6)
15    (19) Merge(0,3,6)
```

Applications - Sorting

► *Sorting*: Divide-and-conquer sorting

```
1 /***** Merging/Combining: One execution example *****/
2 A = [3, 4]          B = [2, 8]
3 1. Assume 4-element array splitted, left = 0; mid = 1, right = 3
4 2. Need to combine and update as a 4-element sorted array,
5     C = [c1, c2, c3, c4], c1 <= c2 <= c3 <= c4
6 3. Find c1 first, subarrays in ascending order, (line 5-10)
7     c1 = A[0], A[0] < B[0], A = [_, 4], B = [2, 8]
8     c1 = B[0], A[0] > B[0], A = [3, 4], B = [_, 8] (wins)
9 4. Find c2, subarrays in ascending order, (line 5-10)
10    c2 = A[0], A[0] < B[1], A = [_, 4], B = [_, 8] (wins)
11    c2 = B[1], A[0] > B[1], A = [3, 4], B = [_, _],
12 5. Find c3, subarrays in ascending order, (line 5-10)
13    c3 = A[1], A[1] < B[1], A = [_, _], B = [_, 8] (wins)
14    c3 = B[1], A[1] > B[1], A = [_, 4], B = [_, _],
15    c4 = B[1] (line 13-14)
16 6. Sorted merge: [B[0], A[0], A[1], B[1]]
```

Applications - Sorting

► *Sorting*: Divide-and-conquer sorting

```
1  /***** Quick Sort *****/
2  void swap(int *a, int *b) {
3      int temp = *a;
4      *a = *b;
5      *b = temp;
6  }
7  int partition(int arr[], int low, int high) {
8      int pivot = arr[high];
9      int i = low - 1;
10     for (int j = low; j < high; j++) {
11         if (arr[j] <= pivot) {
12             i++;
13             swap(&arr[i], &arr[j]);
14         }
15     }
16     swap(&arr[i + 1], &arr[high]); return i + 1;
17 }
```

Applications - Sorting

► *Sorting*: Divide-and-conquer sorting

```
1 /***** Quick Sort *****/
2 void quickSort(int arr[], int low, int high) {
3     if (low < high) {
4         int pi = partition(arr, low, high);
5
6         quickSort(arr, low, pi - 1);
7         quickSort(arr, pi + 1, high);
8     }
9 }
```

Applications - Sorting

► *Sorting*: Divide-and-conquer sorting

```
1 /***** Quick Sort *****/
2 1. s = [4, 3, 8, 1, 0] be the unsorted array
3 2. last place of array, called pivot = s[4] = 0,
4    - Consider the execution with low = 0, high = 4,
5      i = low - 1 = -1 (swap counter).
6    - Find and place the elements less than the pivot contiguously
7      s = [4, 3, 8, 1, 0], i = -1 (no swap)
8 3. Partition index (pi) = i + 1 (0)
9    - Place the pivot element at partion index
10     such that elements to the left are small
11     and elements to the right are large
12     s = [0, 3, 8, 1, 4]
13    - left partition with low = 0, high = pi - 1 = -1
14    - right partition with low = pi + 1 = 1, high = high = 4
15     s = [3, 8, 1, 4]
16 4. Continue
```

Applications - Sorting

► *Sorting*: Divide-and-conquer sorting

```
1 /***** Quick Sort *****/
2 1. s = [_, 3, 8, 1, 4] be the unsorted array
3 2. last place of array, called pivot = s[4] = 4,
4     - Consider the execution with low = 1, high = 4,
5       i = low - 1 = 0 (swap counter).
6     - Find and place the elements less than the pivot contiguously
7       s = [_, 3, 8, 1, 4], i = 1 (self-swap)
8       s = [_, 3, 1, 8, 4], i = 2 (swap)
9 3. Partition index (pi) = i + 1 (3)
10    - Place the pivot element at partition index
11      such that elements to the left are small
12      and elements to the right are large
13      s = [_, 3, 1, 4, 8]
14    - left partition with low = 1, high = pi - 1 = 2
15    - right partition with low = pi + 1 = 4, high = high = 4
16      s_l = [3, 1], s_r = [8] (continue)
```

Applications - Sorting

► *Sorting:* Divide-and-conquer sorting

```
1  /***** Quick Sort *****/
2  1. s = [_, 3, 1, _, _] be the unsorted array
3  2. last place of array, called pivot = s[2] = 1,
4      - Consider the execution with low = 1, high = 2,
5      i = low - 1 = 0 (swap counter).
6      - Find and place the elements less than the pivot contiguously
7      - No swap
8  3. Partition index (pi) = i + 1 (1)
9      - Swap the pivot element and element at partition index 'pi'
10         s = [_, 1, 3, _, _]
11      - left partition with low = 1, high = pi - 1 = 0 (Stops)
12      - right partition with low = pi + 1 = 2, high = 2 (Stops)
13         s_l = [], s_r = [3]
14 4. Stop
```


Applications - Sorting

- ▶ *Sorting*: Divide-and-conquer sorting
- ▶ Recursion stops when left index $\leq$ right index
- ▶ Elements in left partition $\leq$ pivot
- ▶ Elements in right partition $\geq$ pivot

Applications - Sorting

► Algorithm complexity:

Table: Big $\mathcal{O}$ of Sorting Algos.

Algorithm	Big $\mathcal{O}$	DS	Contributor	Name
Bubble Sort	$\mathcal{O}(n^2)$	Array	Nested For	Quadratic
Bubble Sort	$\mathcal{O}(n^2)$	LL	Nested While	Quadratic
Merge Sort	$\mathcal{O}(\log n)$	Array	Split/Combine	Logarithmic
Merge Sort	$\mathcal{O}(n)$	Array	Sort & Combine	Linear
Quick Sort	$\mathcal{O}(n)$	Array	Partition	Linear
Quick Sort	$\mathcal{O}(\log n)$	Array	Placement	Logarithmic

Deque

- ▶ *Deque*: Double-ended Queue (insert from front with delete from back support added to the normal queue)

Deque

```
1 #define MAX 100
2
3 typedef struct {
4     int arr[MAX];
5     int front;
6     int rear;
7 } Deque;
8
9 // Initialize deque
10 void initDeque(Deque *dq) {
11     dq->front = -1;
12     dq->rear = -1;
13 }
14
15 // Check if deque is empty
16 int isEmpty(Deque *dq) {
17     return dq->front == -1;
18 }
```

Deque

```
1 int isFull(Deque *dq) {  
2     // front at 2nd node, fill rear side and front till 1st node  
3     return (dq->front == 0 && dq->rear == MAX - 1) ||  
4         (dq->front == dq->rear + 1); // Think: works both side  
5 }  
6 // Insert at front  
7 void insertFront(Deque *dq, int value) {  
8     if (isFull(dq)) { // list full at both ends  
9         return;  
10    }  
11    if (isEmpty(dq)) { // if empty, insert at start  
12        dq->front = dq->rear = 0;  
13    } else if (dq->front == 0) { // list full at front end  
14        dq->front = MAX - 1; // insert from backside  
15    } else { // front has vacancies  
16        dq->front--;  
17    }  
18    dq->arr[dq->front] = value; // insert at front  
19 }
```

Deque

```
1  // Insert at rear
2  void insertRear(Deque *dq, int value) {
3      if (isFull(dq)) { // list full at both ends
4          return;
5      }
6
7      if (isEmpty(dq)) { // if empty, insert at start
8          dq->front = dq->rear = 0;
9      } else if (dq->rear == MAX - 1) { // list full at back end
10         dq->rear = 0;
11     } else { // back has vacancies
12         dq->rear++;
13     }
14
15     dq->arr[dq->rear] = value; // insert at rear
16 }
```

Deque

```
1 // Delete from front
2 int deleteFront(Deque *dq) {
3     if (isEmpty(dq)) {                // Empty, nothing to delete
4         return -1;
5     }
6     int value = dq->arr[dq->front];    // for return, temporary store
7     // Delete front side and back side, reached a pt, 1 node pending
8     if (dq->front == dq->rear) {
9         dq->front = dq->rear = -1; // delete makes list empty
10        // deleting last node, new front node 0 (min index)
11    } else if (dq->front == MAX - 1) {
12        dq->front = 0;
13    } else {                            // normal delete process
14        dq->front++;
15    }
16    return value;
17 }
```

Deque

```
1 // Delete from rear
2 int deleteRear(Deque *dq) {
3     if (isEmpty(dq)) {                // Empty, nothing to delete
4         return -1;
5     }
6     int value = dq->arr[dq->rear];    // for return, temporary store
7     // Delete front side and back side, reached a pt, 1 node pending
8     if (dq->front == dq->rear) {
9         dq->front = dq->rear = -1; // delete makes list empty
10        // deleting first node, new rear node MAX - 1 (max ind.)
11    } else if (dq->rear == 0) {
12        dq->rear = MAX - 1;
13    } else {
14        dq->rear--;                // delete from rear
15    }
16    return value;
17 }
```